

Data Structure and Algorithm

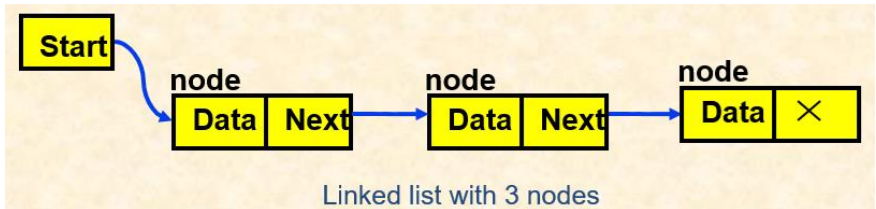
(PMSCS 623P)

Lecture 5

Linked List

❑ Linked Lists

- A link list or one-way list is a linear collection of data elements, called **nodes**, where the linear order is given by **means of pointers**.
- That is, each node is divided into **two parts**: the first part contains the information of the elements, and the second part, called the link field or next pointer field, contains the address of the next node in the list.
- The following figure is a schematic diagram of a linked list with 3 nodes.



Linked List

□ Linked Lists

- The pointer of the last node contains a special value, called the null pointer which is any invalid address.
- A pointer variable, called **START** which contains the address of the first node.
- A special case is the list that has no nodes. Such a list is called the **null list or empty list** and is denoted by the null pointer in the variable **START**.

Linked List

□Memory Representation of the Linked List

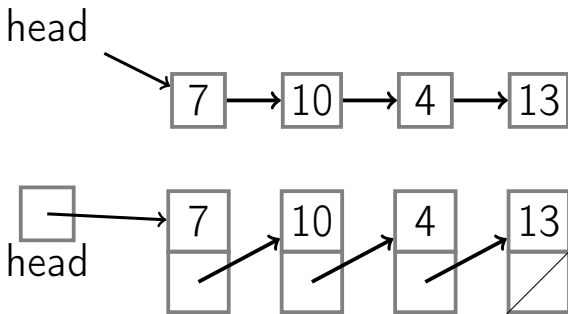
- Let **LIST** be a linked list. Then LIST will be maintained in memory as follows.
- First of all, LIST requires **two linear arrays**—**INFO** and **LINK** such that **INFO[K]** and **LINK[K]** contain, respectively, the information part and the next pointer field of a node of the LIST.
- LIST also requires a variable name such as **START** which contains the location of the beginning of the list and a next pointer sentinel denoted by **NULL** which indicates the end of the list.

Linked List

❑ Distinguish between Array and Link List

- ✓ Both are linear data structures.
- ✓ **Array size is fixed** (dense list or static data structure) but **linked list size is NOT fixed** (dynamic data structure).
- ✓ **Array requires less space** as only information is stored. **Linked list requires more space** as pointers are also stored along with information.
- ✓ Elements are stored in **consecutive memory locations in array**. In **linked list**, elements **may or may not be stored in consecutive** memory locations.
- ✓ **Array needs movements of elements for insertion and deletion**. **Linked list does NOT need movement** of nodes for insertion and deletion.
- ✓ It **can't be extended or reduced** according to requirements in **array**. **Linked list can be extended or reduced** according to requirements.

Singly-Linked List



Node contains:

- key
- next pointer

List API

PushFront(Key) add to front

List API

PushFront(Key)	add to front
Key TopFront()	return front item

List API

PushFront(Key)	add to front
Key TopFront()	return front item
PopFront()	remove front item

List API

PushFront(Key)	add to front
Key TopFront()	return front item
PopFront()	remove front item
PushBack(Key)	add to back
	also known as Append

List API

PushFront(Key)	add to front
Key TopFront()	return front item
PopFront()	remove front item
PushBack(Key)	add to back
Key TopBack()	return back item

List API

PushFront(Key)	add to front
Key TopFront()	return front item
PopFront()	remove front item
PushBack(Key)	add to back
Key TopBack()	return back item
PopBack()	remove back item

List API

PushFront(Key)	add to front
Key TopFront()	return front item
PopFront()	remove front item
PushBack(Key)	add to back
Key TopBack()	return back item
PopBack()	remove back item
Boolean Find(Key)	is key in list?

List API

PushFront(Key)	add to front
Key TopFront()	return front item
PopFront()	remove front item
PushBack(Key)	add to back
Key TopBack()	return back item
PopBack()	remove back item
Boolean Find(Key)	is key in list?
Erase(Key)	remove key from list

List API

<code>PushFront(Key)</code>	add to front
<code>Key TopFront()</code>	return front item
<code>PopFront()</code>	remove front item
<code>PushBack(Key)</code>	add to back
<code>Key TopBack()</code>	return back item
<code>PopBack()</code>	remove back item
<code>Boolean Find(Key)</code>	is key in list?
<code>Erase(Key)</code>	remove key from list
<code>Boolean Empty()</code>	empty list?

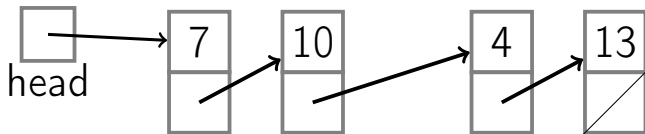
List API

<code>PushFront(Key)</code>	add to front
<code>Key TopFront()</code>	return front item
<code>PopFront()</code>	remove front item
<code>PushBack(Key)</code>	add to back
<code>Key TopBack()</code>	return back item
<code>PopBack()</code>	remove back item
<code>Boolean Find(Key)</code>	is key in list?
<code>Erase(Key)</code>	remove key from list
<code>Boolean Empty()</code>	empty list?
<code>AddBefore(Node, Key)</code>	adds key before node

List API

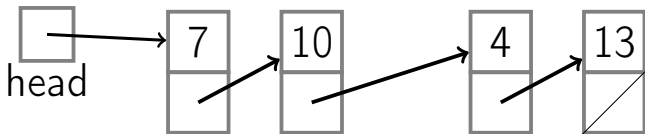
<code>PushFront(Key)</code>	add to front
<code>Key TopFront()</code>	return front item
<code>PopFront()</code>	remove front item
<code>PushBack(Key)</code>	add to back
<code>Key TopBack()</code>	return back item
<code>PopBack()</code>	remove back item
<code>Boolean Find(Key)</code>	is key in list?
<code>Erase(Key)</code>	remove key from list
<code>Boolean Empty()</code>	empty list?
<code>AddBefore(Node, Key)</code>	adds key before node
<code>AddAfter(Node, Key)</code>	adds key after node

Times for Some Operations



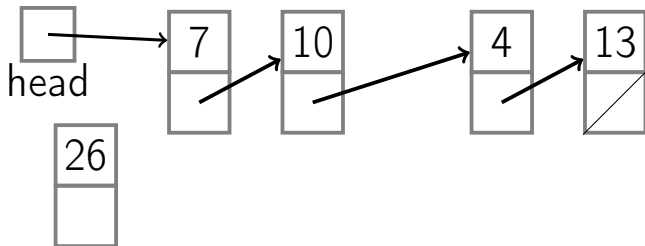
Times for Some Operations

PushFront



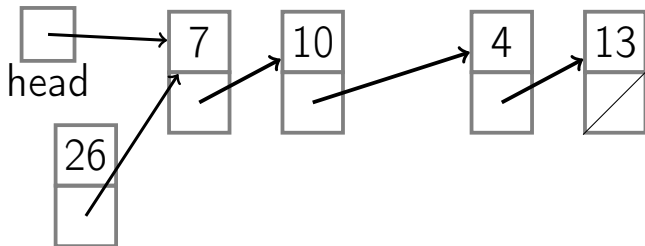
Times for Some Operations

PushFront



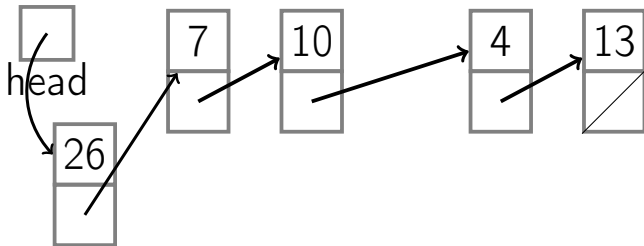
Times for Some Operations

PushFront



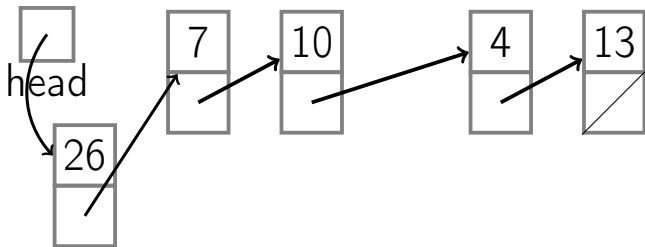
Times for Some Operations

PushFront $O(1)$



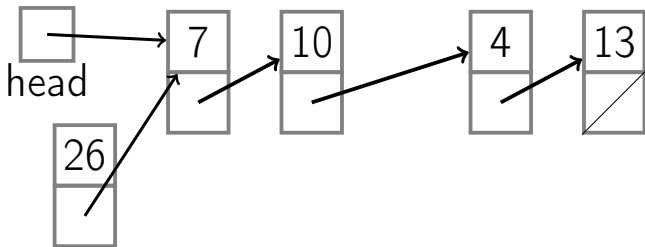
Times for Some Operations

PopFront



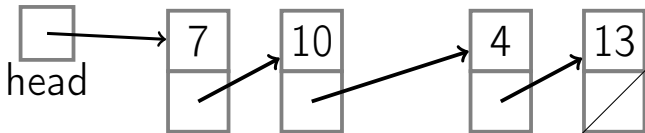
Times for Some Operations

PopFront



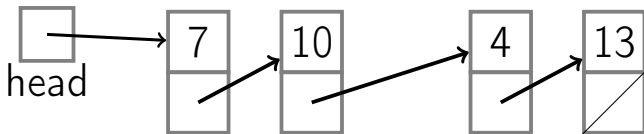
Times for Some Operations

PopFront $O(1)$



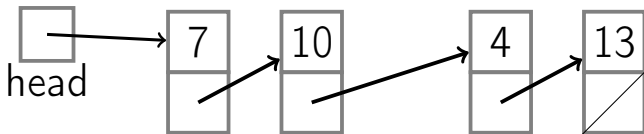
Times for Some Operations

PushBack
(no tail)



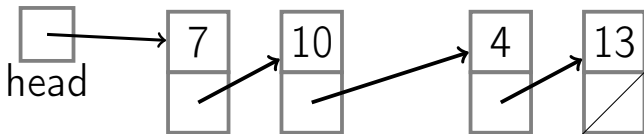
Times for Some Operations

PushBack $O(n)$
(no tail)



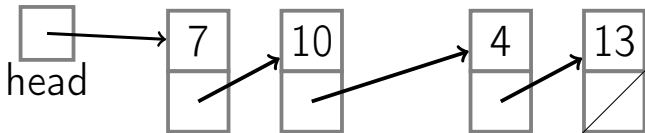
Times for Some Operations

PopBack
(no tail)

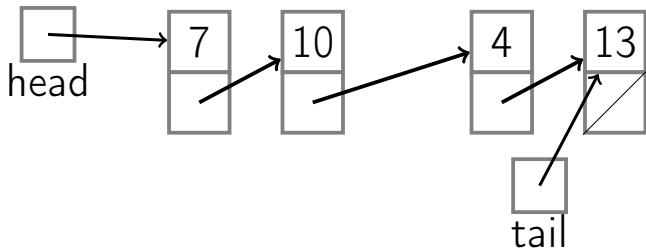


Times for Some Operations

PopBack $O(n)$
(no tail)

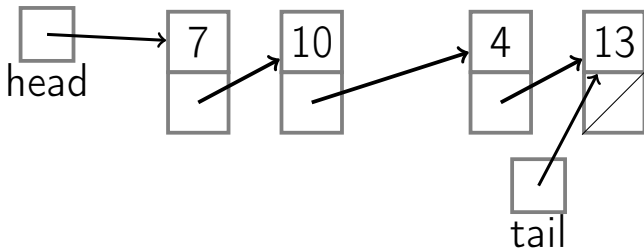


Times for Some Operations



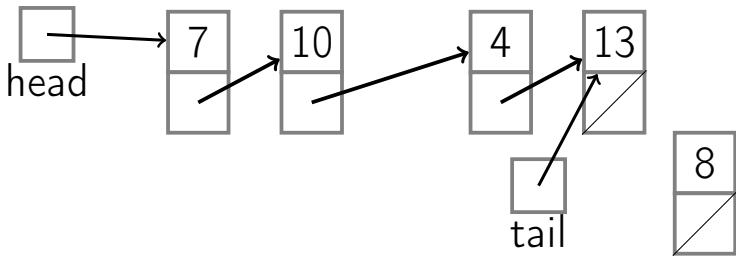
Times for Some Operations

PushBack
(with tail)



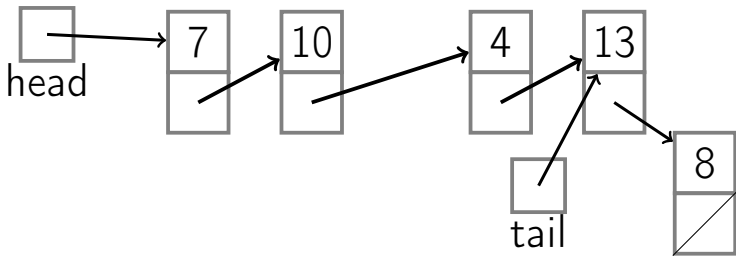
Times for Some Operations

PushBack
(with tail)



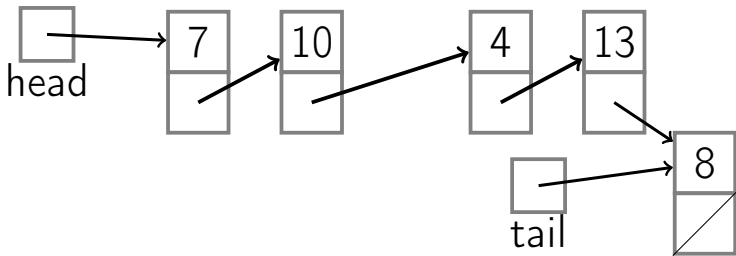
Times for Some Operations

PushBack
(with tail)



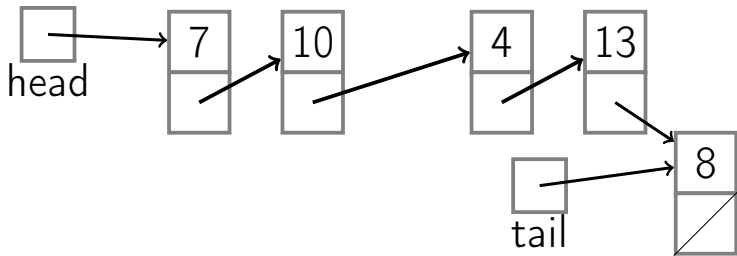
Times for Some Operations

PushBack $O(1)$
(with tail)



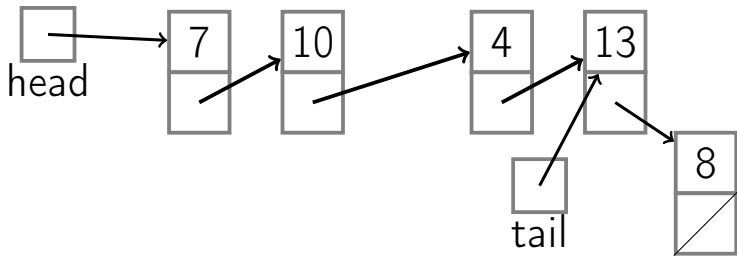
Times for Some Operations

PopBack
(with tail)



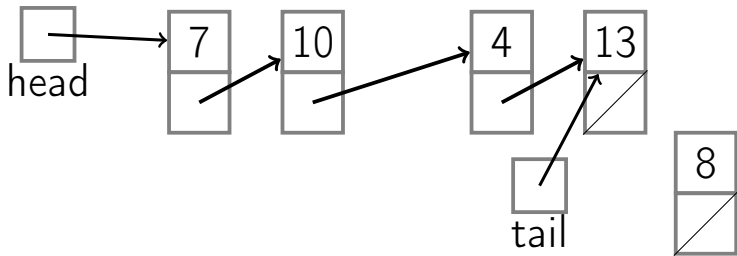
Times for Some Operations

PopBack
(with tail)



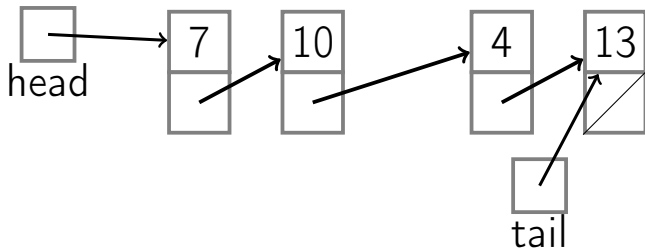
Times for Some Operations

PopBack
(with tail)



Times for Some Operations

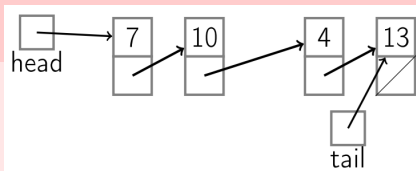
PopBack $O(n)$
(with tail)



Singly-linked List

PushFront(*key*)

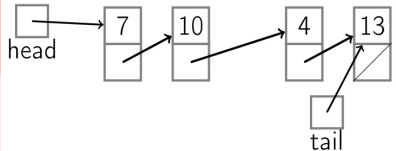
```
node  $\leftarrow$  new node  
node.key  $\leftarrow$  key  
node.next  $\leftarrow$  head  
head  $\leftarrow$  node  
if tail = nil:  
    tail  $\leftarrow$  head
```



Singly-linked List

PopFront()

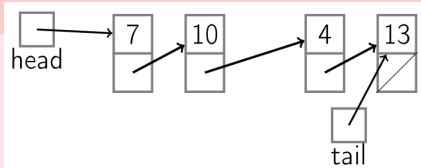
```
if head = nil:  
    ERROR: empty list  
head ← head.next  
if head = nil:  
    tail ← nil
```



Singly-linked List

PushBack(*key*)

```
node ← new node  
node.key ← key  
node.next = nil
```



Singly-linked List

PushBack(*key*)

node $\leftarrow$ new node

node.key $\leftarrow$ *key*

node.next = nil

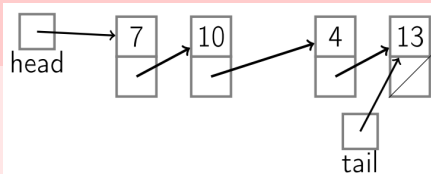
if *tail* = nil:

head $\leftarrow$ *tail* $\leftarrow$ *node*

Singly-linked List

PushBack(*key*)

```
node  $\leftarrow$  new node  
node.key  $\leftarrow$  key  
node.next = nil  
if tail = nil:  
    head  $\leftarrow$  tail  $\leftarrow$  node  
else:  
    tail.next  $\leftarrow$  node  
    tail  $\leftarrow$  node
```



Singly-linked List

PopBack()

Singly-linked List

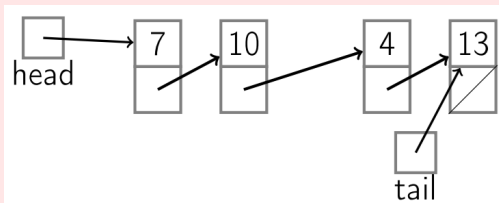
PopBack()

```
if head = nil:  ERROR: empty list
```

Singly-linked List

PopBack()

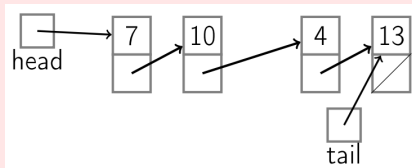
```
if head = nil:  ERROR: empty list  
if head = tail:  
    head  $\leftarrow$  tail  $\leftarrow$  nil
```



Singly-linked List

PopBack()

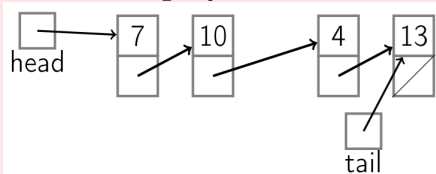
```
if head = nil:  ERROR: empty list
if head = tail:
    head  $\leftarrow$  tail  $\leftarrow$  nil
else:
    p  $\leftarrow$  head
    while p.next.next  $\neq$  nil:
        p  $\leftarrow$  p.next
```



Singly-linked List

PopBack()

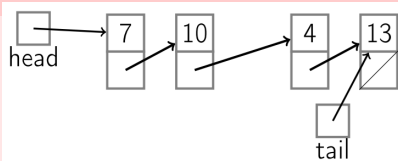
```
if head = nil:  ERROR: empty list
if head = tail:
    head  $\leftarrow$  tail  $\leftarrow$  nil
else:
    p  $\leftarrow$  head
    while p.next.next  $\neq$  nil:
        p  $\leftarrow$  p.next
    p.next  $\leftarrow$  nil; tail  $\leftarrow$  p
```



Singly-linked List

AddAfter(*node*, *key*)

```
node2  $\leftarrow$  new node  
node2.key  $\leftarrow$  key  
node2.next = node.next  
node.next = node2  
if tail = node:  
    tail  $\leftarrow$  node2
```



Singly-Linked List	no tail	with tail
--------------------	---------	-----------

PushFront(Key)	$O(1)$	
----------------	--------	--

Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	

Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	

Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$

Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$

Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$
PopBack()	$O(n)$	

Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$
PopBack()	$O(n)$	
Find(Key)	$O(n)$	

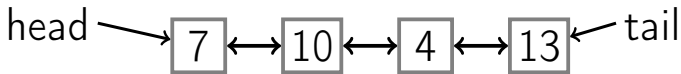
Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$
PopBack()	$O(n)$	
Find(Key)	$O(n)$	
Erase(Key)	$O(n)$	

Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$
PopBack()	$O(n)$	
Find(Key)	$O(n)$	
Erase(Key)	$O(n)$	
Empty()	$O(1)$	

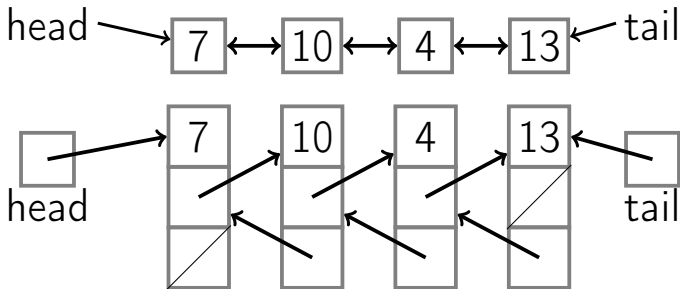
Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$
PopBack()	$O(n)$	
Find(Key)	$O(n)$	
Erase(Key)	$O(n)$	
Empty()	$O(1)$	
AddBefore(Node, Key)	$O(n)$	

Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$
PopBack()	$O(n)$	
Find(Key)	$O(n)$	
Erase(Key)	$O(n)$	
Empty()	$O(1)$	
AddBefore(Node, Key)	$O(n)$	
AddAfter(Node, Key)	$O(1)$	

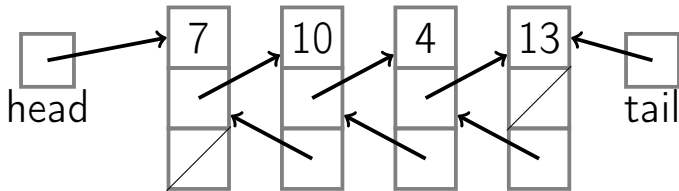
Doubly-Linked List



Doubly-Linked List



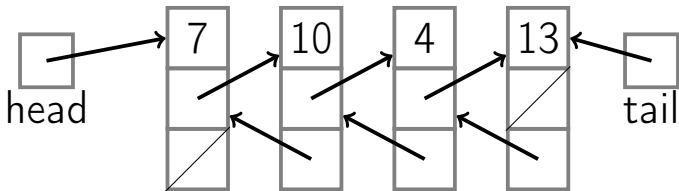
Doubly-Linked List



Node contains:

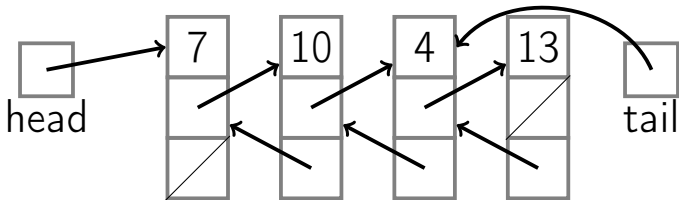
- key
- next pointer
- prev pointer

Doubly-Linked List



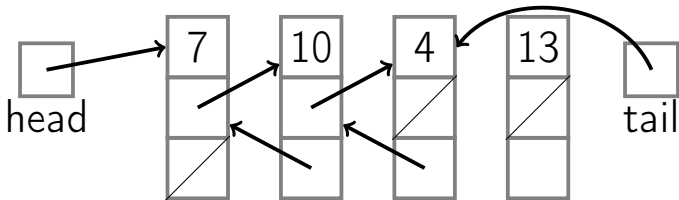
PopBack

Doubly-Linked List



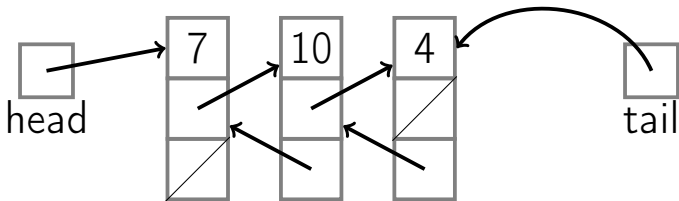
PopBack

Doubly-Linked List



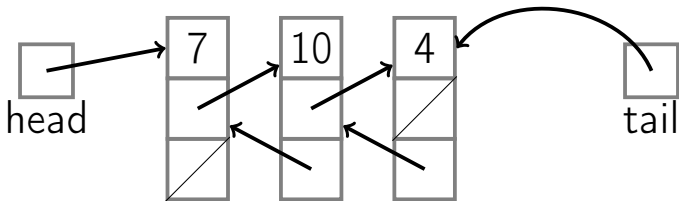
PopBack

Doubly-Linked List



PopBack

Doubly-Linked List



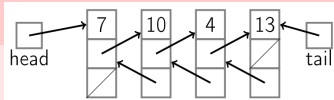
PopBack $O(1)$

Doubly-linked List

PushBack(*key*)

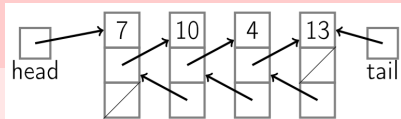
node $\leftarrow$ new node

node.key $\leftarrow$ *key*; *node.next* = nil



Doubly-linked List

PushBack(*key*)



$node \leftarrow \text{new node}$

$node.key \leftarrow key; node.next = \text{nil}$

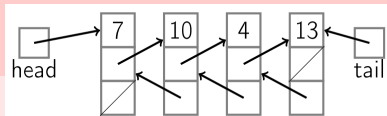
if $tail = \text{nil}$:

$head \leftarrow tail \leftarrow node$

$node.prev \leftarrow \text{nil}$

Doubly-linked List

PushBack(*key*)



node $\leftarrow$ new node

node.key $\leftarrow$ *key*; *node.next* = nil

if *tail* = nil:

head $\leftarrow$ *tail* $\leftarrow$ *node*

node.prev $\leftarrow$ nil

else:

tail.next $\leftarrow$ *node*

node.prev $\leftarrow$ *tail*

tail $\leftarrow$ *node*

Doubly-linked List

PopBack()

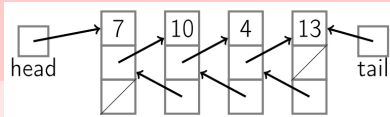
Doubly-linked List

PopBack()

```
if head = nil:  ERROR: empty list
```

Doubly-linked List

PopBack()



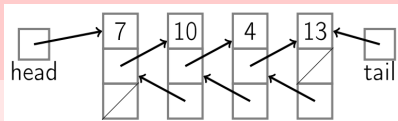
if *head* = nil: ERROR: empty list

if *head* = *tail*:

head $\leftarrow$ *tail* $\leftarrow$ nil

Doubly-linked List

PopBack()



```
if head = nil:  ERROR: empty list
```

```
if head = tail:
```

```
    head  $\leftarrow$  tail  $\leftarrow$  nil
```

```
else:
```

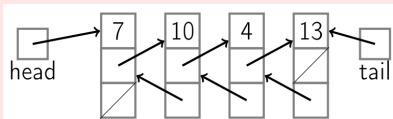
```
    tail  $\leftarrow$  tail.prev
```

```
    tail.next  $\leftarrow$  nil
```

Doubly-linked List

AddAfter(*node*, *key*)

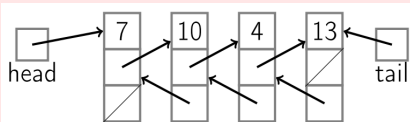
```
node2  $\leftarrow$  new node  
node2.key  $\leftarrow$  key  
node2.next  $\leftarrow$  node.next  
node2.prev  $\leftarrow$  node  
node.next  $\leftarrow$  node2  
if node2.next  $\neq$  nil:  
    node2.next.prev  $\leftarrow$  node2  
if tail = node:  
    tail  $\leftarrow$  node2
```



Doubly-linked List

AddBefore(*node*, *key*)

```
node2  $\leftarrow$  new node  
node2.key  $\leftarrow$  key  
node2.next  $\leftarrow$  node  
node2.prev  $\leftarrow$  node.prev  
node.next  $\leftarrow$  node2  
if node2.next  $\neq$  nil:  
    node2.prev.next  $\leftarrow$  node2  
if head = node:  
    head  $\leftarrow$  node2
```



Singly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$
PopBack()	$O(n)$	
Find(Key)	$O(n)$	
Erase(Key)	$O(n)$	
Empty()	$O(1)$	
AddBefore(Node, Key)	$O(n)$	
AddAfter(Node, Key)	$O(1)$	

Doubly-Linked List	no tail	with tail
PushFront(Key)	$O(1)$	
TopFront()	$O(1)$	
PopFront()	$O(1)$	
PushBack(Key)	$O(n)$	$O(1)$
TopBack()	$O(n)$	$O(1)$
PopBack()	$O(n)$ $O(1)$	
Find(Key)	$O(n)$	
Erase(Key)	$O(n)$	
Empty()	$O(1)$	
AddBefore(Node, Key)	$O(n)$ $O(1)$	
AddAfter(Node, Key)	$O(1)$	

Summary

- Constant time to insert at or remove from the front.

Summary

- Constant time to insert at or remove from the front.
- With tail and doubly-linked, constant time to insert at or remove from the back.

Summary

- Constant time to insert at or remove from the front.
- With tail and doubly-linked, constant time to insert at or remove from the back.
- $O(n)$ time to find arbitrary element.

Summary

- Constant time to insert at or remove from the front.
- With tail and doubly-linked, constant time to insert at or remove from the back.
- $O(n)$ time to find arbitrary element.
- List elements need not be contiguous.

Summary

- Constant time to insert at or remove from the front.
- With tail and doubly-linked, constant time to insert at or remove from the back.
- $O(n)$ time to find arbitrary element.
- List elements need not be contiguous.
- With doubly-linked list, constant time to insert between nodes or remove a node.